

Tutorial: de photextra_pipeline a CIGALE (fotometría + espectroscopía DESI)

Este documento explica, paso a paso, cómo instalar el CIGALE usado en este proyecto, cómo se registraron los filtros propios (CHANCES continuum + Legacy/DECam), cómo funciona la espectroscopía nativa DESI (`use_spectro`), y cómo usar el `cigale_input.txt` que arma `scripts/build_cigale_input.py` a partir de los productos de `photextra_pipeline` .

Todos los paths de ejemplo son los reales de esta máquina; ajústelos si cambia la ubicación del código.

1. Instalación de CIGALE

Se usa un fork de CIGALE (`cigale_spec`) con soporte de ajuste espectroscópico nativo (`use_spectro=True`), que **no** es el CIGALE oficial estándar (ese no trae esta funcionalidad lista para usar de la misma forma). El código fuente vive en:

```
/home/polo/Escritorio/PHD/code/cigale-spec/
```

1.1 Crear el entorno conda

```
conda create -n cigale python=3.13 -y
conda activate cigale
```

1.2 Instalar dependencias + el paquete

```
cd /home/polo/Escritorio/PHD/code/cigale-spec
pip install .
```

Esto instala `pcigale` , `pcigale_filters` y `pcigale_plots` (además de sus dependencias: `astropy`, `configobj`, `matplotlib`, `numpy`, `rich`, `scipy`). Confirmar:

```
python -c "import pcigale; print(pcigale.__file__)"
pcigale --help
```

1.3 Notas

- Se ajustaron los threads de BLAS a 1 en otros paquetes del proyecto (XspectraFit) para evitar contención de CPU en paralelización — no aplica directamente a CIGALE, pero si se ejecuta todo en el mismo proceso Python, hay que tenerlo en cuenta.
- `cores = N` en `pcigale.ini` reparte el cómputo de modelos en N procesos. Usar el número real de cores de la máquina.

2. Filtros propios registrados

CIGALE trae un catálogo de filtros estándar (GALEX, SDSS, WISE, 2MASS, etc.) pero **no** trae las bandas custom que usa este proyecto:

1. **10 filtros continuum CHANCES** (tophats angostos, diseñados para evitar líneas de emisión y bandas telúricas, usados para hacer fotometría sintética directamente del espectro DESI).
2. **Curvas reales DECam/Legacy Survey g/r/z** (namespaced aparte para no confundirlas con un SDSS genérico).

Los `.dat` ya generados están en `filters/` junto a este tutorial, listos para copiar/registrar en cualquier instalación nueva de CIGALE. También están los scripts que los generan, por si cambian los filtros CHANCES o aparecen nuevas bandas.

2.1 Filtros continuum CHANCES (`chances.*`)

Fuente de verdad: `photextra_pipeline/data/chances_continuum_filters.csv` (columnas `name`, `lambda_min`, `lambda_max`, `enabled`, `notes` , en Å). Son tophats puros (transmisión 1 dentro del rango, 0 fuera), pensados para sintetizar fotometría de banda ancha directo del espectro sin caer en líneas de emisión fuertes ni en bandas telúricas (ver columna `notes` del CSV para el criterio de cada uno).

Filtros habilitados actualmente (`enabled=1`):

Nombre CIGALE	λ_{min} (Å)	λ_{max} (Å)	Motivo
<code>chances.M3992</code>	3750	4230	continuo azul del salto de 4000 Å
<code>chances.M4542</code>	4440	4650	evita H β /OIII
<code>chances.M5200</code>	5100	5450	lado azul de Mgb
<code>chances.M5600</code>	5450	5800	continuo Mgb/Fe, lado rojo
<code>chances.N6097</code>	6000	6200	antes de NaD

Nombre CIGALE	$\lambda_{\min}$ (Å)	$\lambda_{\max}$ (Å)	Motivo
chances.N6350	6200	6480	evita el complejo H α /NII/SII
chances.N6908	6900	7020	evita telúrica O2-B
chances.07473	7200	7580	evita telúrica O2-A
chances.08000	7810	8090	evita H2O moderada
chances.08550	8390	8800	cubre el triplete de CaII (real, no telúrica)

Regenerar/registrar (por ejemplo si se agrega un filtro nuevo al CSV):

```
conda activate cigale
cd photextra_pipeline/scripts
python make_cigale_filters.py          # lee el CSV, escribe .dat, registra
# --no-register -> solo escribe los .dat, no toca la DB de CIGALE
# --include-disabled -> también genera los que tienen enabled=0
```

Internamente escribe cada filtro como un `.dat` de 3 líneas de cabecera + tabla wavelength/transmisión:

```
# chances.M3992
# photon
# CHANCES continuum tophat 3750-4230A -- ...
3745.00 0.000000
3750.00 1.000000
...
```

Tipo `photon` (no `energy`): así integra igual que `xpctrafit` ($\langle f \rangle = \int f \cdot T \cdot \lambda \, d\lambda / \int T \cdot \lambda \, d\lambda$), para que la fotometría sintética del pipeline y los flujos de modelo de CIGALE se calculen con la misma convención.

2.2 Filtros reales DECam/Legacy (`chances_legacy.*`)

Curvas de throughput real del sistema DECam (no genéricas), tomadas de `hostphot` (la misma librería que usa `photextra_pipeline.downloader` para bajar los cutouts de Legacy Survey — son las curvas correctas porque son las mismas contra las que está definida la fotometría real):

```
hostphot/filters/LegacySurvey/DECAM_{g,r,z}.dat
```

Regenerar/registrar:

```
conda activate cigale
cd photextra_pipeline/scripts
python make_cigale_legacy_filters.py
# --srcdir PATH -> si hostphot está en otro conda env
# --no-register -> solo escribe los .dat
```

Tipo `energy` (no `photon`): es la convención correcta para una curva de respuesta real de instrumento integrada contra F_λ , igual que el resto de filtros reales ya en la base (galex.FUV, wise.W1, sdss.*).

2.3 Verificar qué quedó registrado

```
pcigale-filters list | grep -i "chances\|decam"
```

Si se necesita reinstalar CIGALE desde cero, ejecutar ambos scripts (`make_cigale_filters.py` y `make_cigale_legacy_filters.py`) antes de armar cualquier `pcigale.ini` que los use — si un filtro no está registrado, CIGALE falla al validar la configuración (`Unknown filter ...`).

3. Espectroscopía nativa DESI (`use_spectro=True`)

Este es el método recomendado (ver comparación de métodos, sección 5): en vez de sintetizar fotometría de banda angosta, se ajusta el **espectro completo** directamente. Requiere 3 piezas.

3.1 Espectro cacheado → FITS "prism"

`photextra_pipeline` cachea el espectro DESI en `.npz` (`wave` en Å, `flux` en 10^{-17} erg/s/cm²/Å, `ivar`). CIGALE (`read_prism()`) espera un FITS con tabla binaria de 3 columnas: `WAVELENGTH` (micron), `FLUX` (mJy), `FLUX_ERROR` (mJy). Conversión:

```
python scripts/convert_desi_to_prism_fits.py desi_<tid>.npz desi_spec_<tid>.fits
```

Clave física de la conversión ($F_\lambda \rightarrow F_\nu$ en mJy):

```
conv = wave**2 / C_AA_S * 1e26      # (erg/s/cm2/A) -> mJy
fnu  = flam * conv
```

`build_cigale_input.py` (sección 4) hace esto automáticamente, no hace falta ejecutarlo a mano salvo para depuración.

3.2 Archivo de dispersión espectral

CIGALE arma internamente una grilla fija de 512 puntos (`new_wavegrid()` , paso $\lambda / (2.2 \cdot R)$) a partir de un archivo de 3 columnas `WAVELENGTH` `dlambda` `R` . La resolución nativa de DESI ($R \sim 4500\text{--}12000$, ~ 0.8 Å/pix) en 512 puntos solo cubriría ~ 185 Å — por eso se calcula un **R efectivo constante** que fuerza los 512 puntos a cubrir todo el rango observado ($\sim 3600\text{--}9824$ Å):

```
python scripts/build_desi_dispersion_file.py desi_<tid_cualquiera>.npz desi_disp.txt
```

Un solo `desi_disp.txt` sirve para todos los targets (misma instrumentación DESI, aproximación razonable).

3.3 Columnas extra en el `data_file`

Además de la fotometría de banda ancha, el `data_file` necesita 3 columnas para activar el modo espectro (nombres específicos de este fork — `spectrum` / `mode` / `norm`, **no** los `spec_name` / `disperser` / `norm_method` del CIGALE oficial):

```
id redshift galex.FUV galex.FUV_err ... WISE2_err spectrum mode norm
2842599849197568 0.029661 ... /ruta/desi_spec_2842599849197568.fits desi wave
```

3.4 `pcigale.ini`

```
data_file = cigale_input.txt
use_spectro = True
spectral_res_file = desi_disp.txt
sed_modules = sfhdelayed, bc03, nebular, dustatt_modified_starburst, dl2014, redshifting

[analysis_params]
bands = galex.FUV, galex.NUV, chances_legacy.DECam_g, chances_legacy.DECam_r, chances_legacy.DECam_z, WISE1, WISE2
```

`[analysis_params].bands` lleva **solo** la fotometría de banda ancha — nunca las columnas del espectro, esas las maneja `use_spectro` internamente.

3.5 Trampa conocida: la key `bands` de nivel superior

Hay una key `bands = ...` **fuera de cualquier sección** (antes de `[sed_modules_params]`) que este fork sí usa en serio (`pcigale/session/configuration.py:274`), distinta de `[analysis_params].bands`. Se recalcula sola con `pcigale genconf` (intersección columnas-del-`data_file` $\cap$ filtros registrados en la DB). Si queda una versión vieja (por ejemplo de un experimento anterior con 512 pseudo-filtros `desi_Band_*`, ver `register_desi_pseudo_filters.py` — un método alternativo que se probó y **no** se terminó usando), falla con:

```
Exception: desi_Band_000 to be taken in the fit but not present in the observation table.
```

Fix: dejar que `pcigale genconf` la recalcule, o setearla a mano igual a `[analysis_params].bands` + sufijo `_err` en cada banda.

Atención: `pcigale genconf` reinicia **todos** los parámetros de `[sed_modules_params]` a sus valores por defecto. Si se ejecuta `genconf` después de armar la grilla de parámetros (`tau_main`, `age_main`,

E_BV_lines, etc.), hay que volver a aplicarla después — nunca antes de ejecutar `pcigale run`.

4. De productos de photextra a `cigale_input.txt`

`photextra_pipeline` en sí **no** exporta un `cigale_input.txt` directamente — genera, por target, dos productos que sí contienen todo lo necesario:

```
{RUN_DIR}/{tid}/products/{tid}_combined.csv    # fotometría total+fibra, z, líneas de emisión
{RUN_DIR}/{tid}/spectroscopy/desi_{tid}.npz    # espectro DESI crudo cacheado
```

El puente entre esos productos y CIGALE es `scripts/build_cigale_input.py` (generaliza lo que se armó a mano en las corridas piloto). Arma **los dos formatos de `cigale_input.txt`** a la vez (fotometría+filtros+líneas, y espectro nativo), para poder compararlos.

4.1 Qué columnas usa de `{tid}_combined.csv`

- `z` → columna `redshift`
- `phot_{survey}_flux_mjy` / `phot_{survey}_flux_err_mjy` (`survey = GALEX_FUV, GALEX_NUV, Legacy_g/r/z, WISE_W1/W2`) → fotometría total de banda ancha, con piso de error del 10% sumado en cuadratura.
- `line_{Halpha,Hbeta,OIII5007,NII6584}_flux_scaled` / `..._flux_err_scaled` → líneas de emisión, en unidades de 10^{-17} erg/s/cm² (igual que el propio espectro), convertidas a W/m² (factor 10^{-20}) para el formato que CIGALE espera en `line.*`.

4.2 Qué hace con `desi_{tid}.npz`

- Método **filtros**: integra el espectro contra los 10 filtros CHANCES (vía `spectrafit.filters.integrate_filters`), corrige de apertura-fibra a apertura-total con `spec_normalization.py` (ajustado contra Legacy g/r/z), agrega piso de error del 10%.
- Método **espectro nativo**: convierte el `.npz` a FITS prism (ver §3.1) y arma la fila con `spectrum / mode / norm`.

4.3 Uso

```
conda activate cigale
cd /home/polo/Escritorio/PHD/CHANCES/MKW8_full_run    # o el RUN_DIR que corresponda
python /home/polo/Escritorio/PHD/code/photextra_pipeline/scripts/build_cigale_input.py <N> [out_root]
```

- `<N>` : cuántos targets tomar (los primeros N que tengan **ambos** productos completos — fotometría y espectroscopía).
- `[out_root]` : directorio de salida (default: el mismo `RUN_DIR`).

Escribe:

```
{out_root}/cigale_N{N}_filters/cigale_input.txt    # método filtros+broadband+líneas
{out_root}/cigale_N{N}_spectro/cigale_input.txt    # método espectro nativo
{out_root}/cigale_N{N}_spectro/desi_spec_{tid}.fits # un FITS por target
```

Faltan a mano (no los genera el script, copiarlos de una corrida piloto previa o rehacerlos con §3.2/§2):

- `pcigale.ini` (uno por método — copiar de una corrida piloto y ajustar `data_file`)
- `desi_disp.txt` (solo para el método espectro nativo)

Ejemplo mínimo de flujo completo:

```
conda activate cigale
cd /home/polo/Escritorio/PHD/CHANCES/MKW8_full_run

# 1. armar los data_file para 50 targets
python ../scripts/build_cigale_input.py 50

# 2. copiar configs base ya validadas
cp cigale_pilot_3targets_v2_lines/pcigale.ini{,.spec}    cigale_N50_filters/
cp cigale_pilot_3targets_desi_spectro/pcigale.ini{,.spec} cigale_N50_spectro/
cp cigale_pilot_3targets_desi_spectro/desi_disp.txt      cigale_N50_spectro/

# 3. correr
cd cigale_N50_spectro && pcigale run
```

5. Filtros → carpeta lista para reusar

Todo lo necesario para levantar los filtros en una instalación nueva de CIGALE está en `filters/` junto a este documento:

```
filters/
├─ chances_continuum_filters.csv    # tabla fuente (10 tophats CHANCES)
├─ M3992.dat ... 08550.dat          # los 10 .dat ya generados (chances.*)
└─ DECam_g.dat, DECam_r.dat, DECam_z.dat # curvas reales (chances_legacy.*)
```

Para registrarlos en una instalación nueva:

```
conda activate cigale
pcigale-filters add filters/M3992.dat filters/M4542.dat filters/M5200.dat \
```

```
filters/M5600.dat filters/N6097.dat filters/N6350.dat filters/N6908.dat \
filters/07473.dat filters/08000.dat filters/08550.dat \
filters/DECam_g.dat filters/DECam_r.dat filters/DECam_z.dat
```

6. Resumen de resultados de la comparación de métodos (446 targets MKW8)

- **Espectro nativo** ajusta mejor en general (χ^2_{red} mediano 2.05 vs 3.45 del método filtros), menos catastróficos (3.8% vs 21.3% con $\chi^2 > 10$).
- Acuerdo de masa estelar entre métodos: excelente (offset mediano -0.06 dex, dispersión 0.21 dex).
- Tiempo: filtros ~ 10.6 s / espectro ~ 35 s para 446 targets con la grilla SFH original (`sfhdelayed` , 17280 modelos/z) — trivial en ambos casos.
- **Ojo con** `sfhdelayedbq` (SFH delayed+quench, para grillas con `age_bq` / `r_sfr`): el costo de `bc03.convolve()` escala con el largo del arreglo de edad (`age_main` , resolución fija de 1 Myr). Si `age_main` se concentra en edades viejas (11000-13000 Myr) para **todos** los combos, el costo por modelo se dispara ~ 20 - $25\times$ vs una grilla log-espaciada de edades. Recortar `r_sfr` / `age_bq` a subconjuntos chicos si se necesita este módulo.